

Patrón: Strategy

Patrón de comportamiento (POO)

Fuente principal:

<https://refactoring.guru/es/design-patterns/strategy>

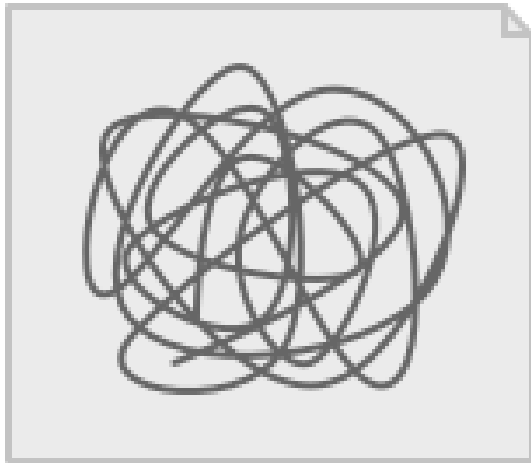
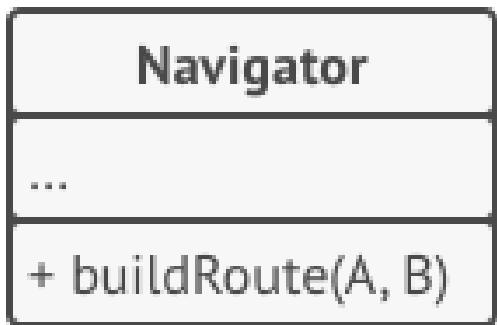
Propósito

Strategy es un patrón de diseño de comportamiento que te permite:

- Definir una **familia de algoritmos**.
- Encapsular cada algoritmo en una clase.
- Hacerlos **intercambiables** (en tiempo de ejecución).

Idea clave: sustituir *if/else* grandes por **polimorfismo + composición**.





Problema

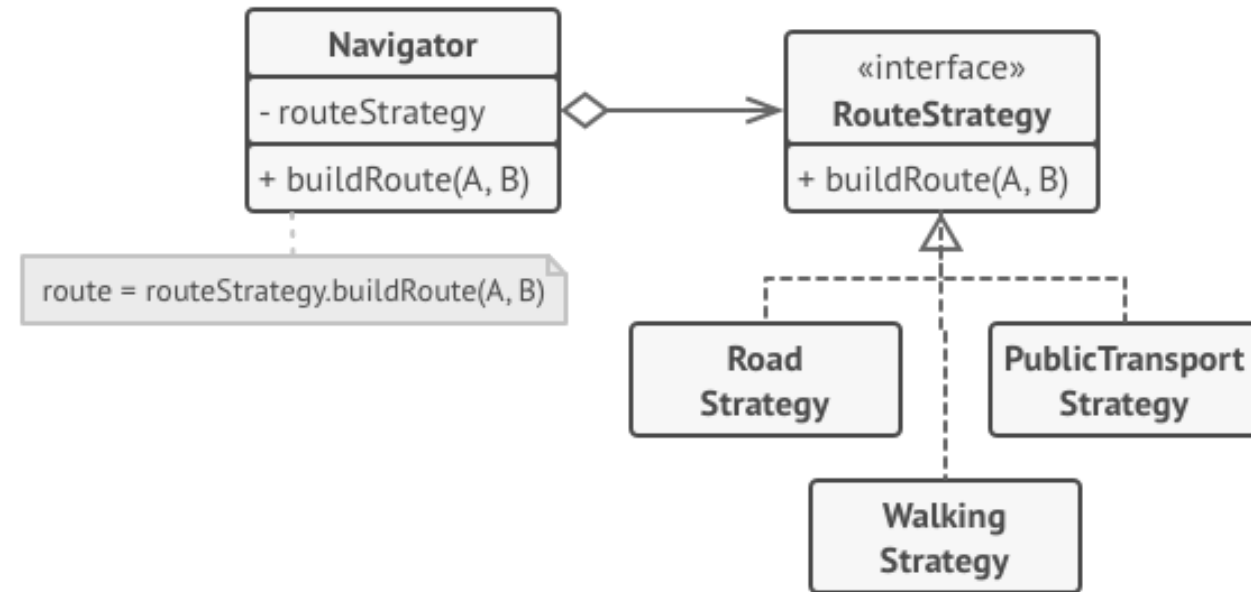
Cuando una clase hace “lo mismo” de muchas formas:

- Empieza soportando **una** variante (p. ej. rutas en coche).
- Luego se añaden nuevas (a pie, transporte público, bici...).
- La clase central crece, se llena de condicionales y se vuelve:
 - difícil de mantener
 - difícil de extender
 - fuente de conflictos en trabajo en equipo

Solución

El patrón propone separar el “cómo” (algoritmo) del “cuándo/para qué” (contexto):

- **Contexto:** mantiene una referencia a una estrategia.
- **Estrategia:** interfaz común del algoritmo.
- **Estrategias concretas:** implementan variantes.
- **Cliente:** elige/inyecta la estrategia y puede cambiarla en runtime.

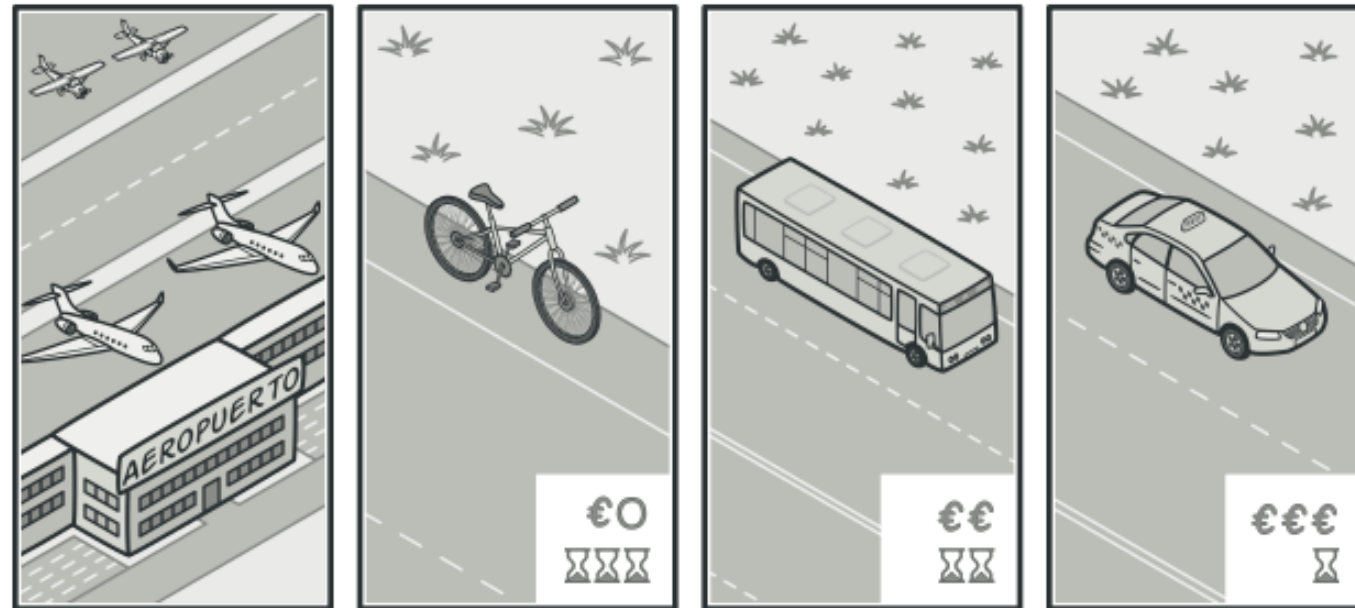


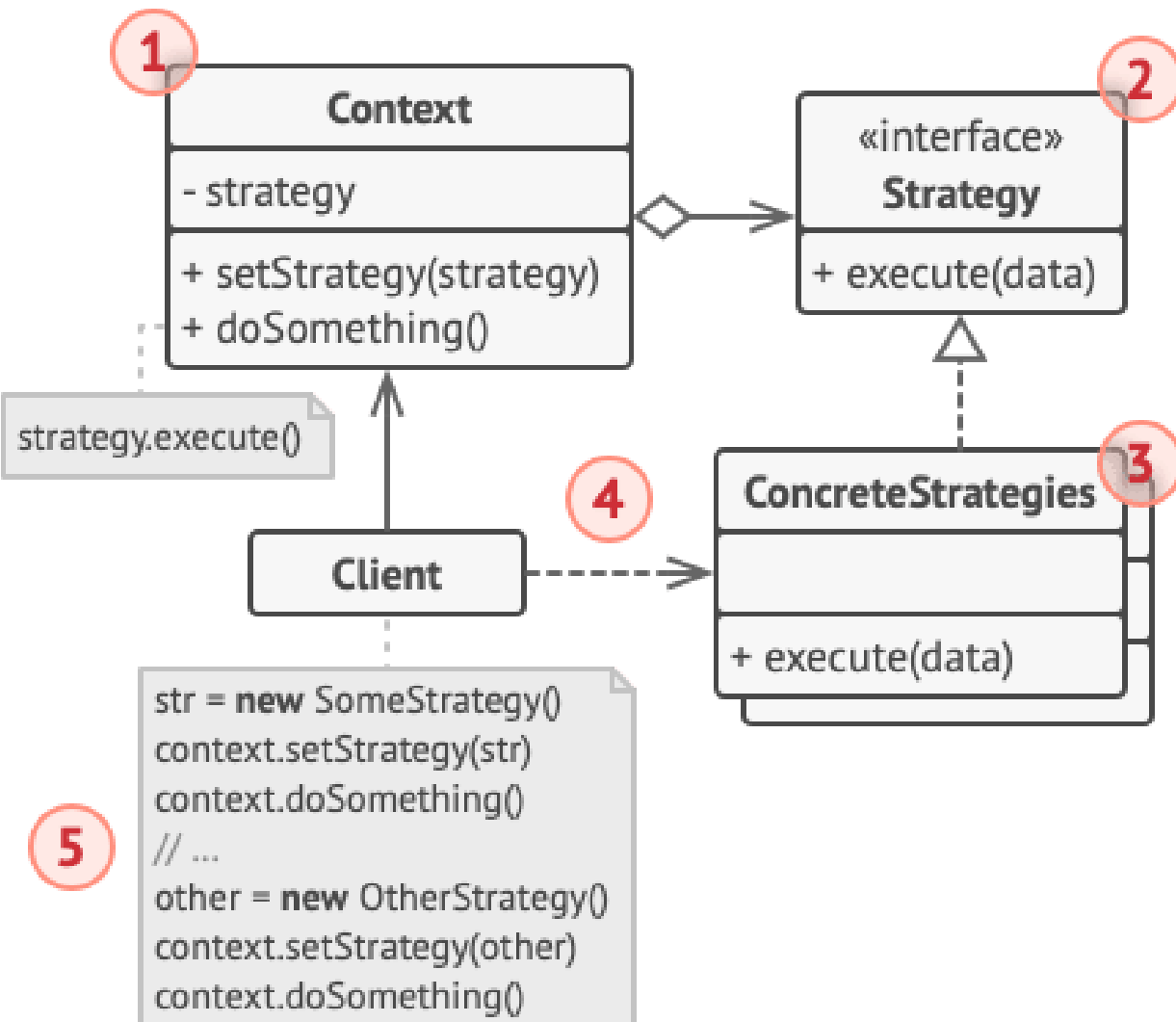
Analogía en el mundo real

Llegar al aeropuerto:

- Autobús
- Taxi
- Bicicleta

Mismo objetivo, **estrategias distintas** según coste, tiempo o preferencias.

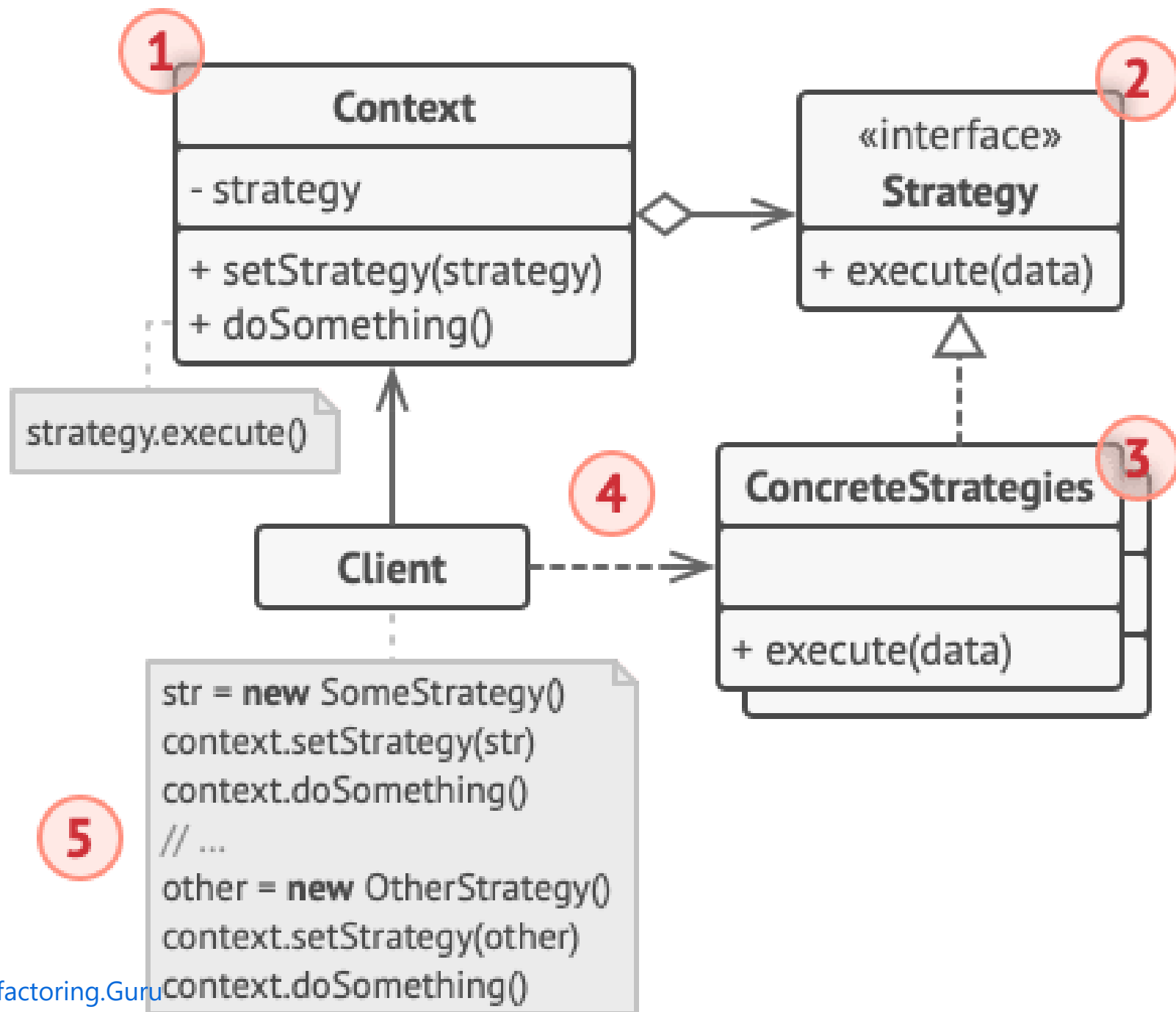




Estructura

Roles típicos:

1. **Contexto**: delega el algoritmo en la estrategia actual.
2. **Strategy**: interfaz con la operación del algoritmo.
3. **ConcreteStrategy**: implementaciones alternativas.
4. **Cliente**: configura el contexto y puede cambiar la estrategia.



Aplicabilidad

Úsalo cuando:

- Necesites **cambiar un algoritmo** dentro de un objeto en tiempo de ejecución.
- Tengas muchas clases similares que solo difieren en *cómo* hacen algo.
- Quieras aislar lógica de negocio de detalles de implementación.
- Tengas un condicional enorme que selecciona variantes del mismo algoritmo.

Pros y contras

Pros

- Cambias algoritmos en runtime.
- Aíslas implementación (menos acoplamiento).
- Sustituyes herencia por **composición**.
- Principio **abierto/cerrado**: nuevas estrategias sin tocar el contexto.

Contras

- Si hay pocas variantes y raramente cambian, puede ser sobreingeniería.
- El cliente debe conocer diferencias para elegir bien.
- En lenguajes con funciones de orden superior, a veces basta con lambdas.

Código Java (puntos clave)

Basado en `code/es/uva/poo/strategy` .

- `Navegador` (Context) **delegará** el algoritmo.
- `EstrategiaRuta` (Strategy) define la operación común.
- Las estrategias concretas implementan variantes.

```
// Strategy
public interface EstrategiaRuta {
    void construirRuta(String origen, String destino);
}

// Context
public class Navegador {
    private EstrategiaRuta estrategia;

    public Navegador(EstrategiaRuta estrategia) {
        this.estrategia = estrategia;
    }

    public void setEstrategia(EstrategiaRuta estrategia) {
        this.estrategia = estrategia;
    }
}
```

Código Java (cambio de estrategia en runtime)

```
public class Demo {  
    public static void main(String[] args) {  
        String origen = "Valladolid";  
        String destino = "Madrid";  
  
        Navegador navegador = new Navegador(new EstrategiaRutaCoche());  
        navegador.construirRuta(origen, destino);  
  
        navegador.setEstrategia(new EstrategiaRutaBicicleta());  
        navegador.construirRuta(origen, destino);  
  
        navegador.setEstrategia(new EstrategiaRutaTransportePublico());  
        navegador.construirRuta(origen, destino);  
    }  
}
```